

NATIONAL TEACHERS COLLEGE

School of Arts, Sciences, and Technology – BSIT Program

Course Code: NTC_CC105 – Data Structure w/ Lab

Academic Year: 2026–2027 | Semester: 1st Semester

Instructor: Prof. Darwin C. Vargas, MIT

LABORATORY ACTIVITY SHEET – WEEK 5

Topic: Linear Architecture Synthesis & Peer Code Audit

Course Title	NTC_CC105 – Data Structure w/ Lab
Target Program	Bachelor of Science in Information Technology (BSIT)
Programming Language	C++ (ANSI/ISO Standard C++17 or higher)
Execution Modality	Individual Code Audit & Pair Refactoring / Peer Defense
Course Outcomes Addressed	C02: Implement linear and non-linear data structures in modular C++ code with dynamic memory handling. C05: Select, justify, and integrate appropriate data structures for solving complex software architecture challenges.
Target Milestone	Pre-Milestone Synthesis (Preparation for Milestone 1 Practical Defense)
Primary Deliverable Output	Module Draft: Peer-Audited C++ Linear Data Architecture Subsystem

1. LABORATORY TITLE

Peer-Audited C++ Linear Data Architecture Subsystem: Quality Audit, Unit Testing, and Architectural Refactoring

2. INTRODUCTION AND BACKGROUND

As software systems scale, structural flaws, memory mismanagement, and unoptimized algorithmic lookup routines degrade overall system performance and stability. Prior to Week 5, you explored core linear data architecture concepts: Abstract Data Types (ADTs), manual dynamic memory allocation (`new`, `delete`), dynamic array manipulation, bounds checking, and multi-dimensional matrix operations.

This laboratory activity acts as a synthesis point before your Milestone 1 (Prelims) Practical Defense. You will adopt the role of a Junior Software Engineer assigned to audit, benchmark, refactor, and harden a legacy C++ linear data storage module. Rather than rewriting code from scratch, professional software engineering requires you to audit existing code bases, locate memory leaks and logical flaws, construct rigorous test cases, compute theoretical Big-O time and space complexities, and execute peer code reviews.

3. LEARNING OBJECTIVES

By completing this laboratory activity, you should be able to:

- **Analyze** an existing C++ linear data structure implementation to identify memory leaks, logical errors, buffer overflows, and architectural inefficiencies.
- **Perform** a structured peer code review utilizing an industry-aligned quality audit checklist.
- **Design and execute** a unit testing suite comprising normal, boundary, and edge-case test vectors.
- **Evaluate** the theoretical time ($O(1)$, $O(n)$, $O(n^2)$) and space complexity of linear routines before and after optimization.
- **Refactor** legacy C++ code to enforce modularity, proper variable scope, RAII dynamic memory safety, and clean coding standards.
- **Document** code modifications, testing outcomes, and repository history in preparation for Milestone 1 submission.

4. CO / CLO ALIGNMENT

C02: Implement linear data structures in modular C++ code with dynamic memory handling.	Audit and fix raw pointer arithmetic, <code>new[]/delete[]</code> lifecycle, dynamic array resizing, and memory leak vulnerabilities.	Students debug legacy dynamic memory allocations and replace dangerous operations with dynamic array management routines.
C05: Select, justify, and integrate appropriate data structures for complex software challenges.	Evaluate linear array limitations against dynamic allocation needs, refactoring unoptimized search/shift operations.	Students analyze algorithmic trade-offs, document Big-O complexity shifts, and defend refactoring choices during peer audit.

5. REQUIRED HARDWARE AND SOFTWARE

1. **Hardware:** Personal Computer running Windows 10/11, macOS, or Linux (Minimum 4GB RAM, Dual-Core Processor).
2. **Compiler & Environment:** GCC / Clang / MSVC supporting C++17 standard or higher.
3. **Integrated Development Environment (IDE):** VS Code (with C/C++ Extension Pack), CLion, or Code::Blocks.
4. **Diagnostic Tools:** Valgrind, GCC AddressSanitizer (`-fsanitize=address`), or Visual Studio Memory Leak Detector.
5. **Version Control:** Git command-line client or GitHub Desktop.

6. PREREQUISITE KNOWLEDGE

Before attempting this activity, you must be comfortable with the following concepts from Weeks 1 through 4:

- Dynamic memory allocation and deallocation using `new` and `delete` / `delete[]`.
- Pointer arithmetic, dereferencing, and distinguishing between stack and heap memory.
- Static vs. dynamic array traversal, insertion, deletion with element shifting, and bounds enforcement.
- Multi-dimensional matrix layout in memory (row-major vs. column-major ordering).

- Modular C++ functions, pass-by-value vs. pass-by-reference (&), and struct/class encapsulation.

7. LABORATORY SCENARIO

You have recently joined **NovaTech Software Solutions** as a Junior C++ Systems Developer. The engineering team is preparing to ship Milestone 1 of their flagship platform: an engine that manages linear batch records and multi-dimensional sensor grid data.

A legacy module, `LegacyLinearSubsystem.cpp`, was written by a former intern. While it compiles under default settings, quality assurance (QA) reports frequent system crashes under heavy loads, memory leaks, invalid array index out-of-bound access, extremely slow processing times on large records, and unreadable monolithic code functions.

Your lead architect has assigned you to audit this legacy module. You must **audit, test, analyze, document, refactor, and conduct a peer code review** on the codebase. You are strictly forbidden from discarding the file and rewriting it without documentation—every change must be justified through data, unit tests, and Big-O efficiency analysis.

Legacy Codebase for Ingestion (`LegacyLinearSubsystem.cpp`)

```
#include <iostream>
#include <string>
using namespace std;

// Legacy Linear Record Subsystem
// Flawed implementation for auditing

int* dataArray = NULL;
int currentCount = 0;
int maxCapacity = 5;

void add_item(int val) {
    if (currentCount == 0) {
        dataArray = new int[maxCapacity];
    }
    if (currentCount >= maxCapacity) {
        // Double capacity without freeing old memory
        int* temp = new int[maxCapacity * 2];
        for (int i = 0; i < maxCapacity; i++) {
```

```

    temp[i] = dataArray[i];
}
dataArray = temp; // Memory leak! Lost pointer to old array
maxCapacity = maxCapacity * 2;
}
dataArray[currentCount] = val;
currentCount++;
cout << "Added item: " << val << endl;
}

void remove_item_at(int idx) {
    // Missing upper bound check!
    if (idx < 0) {
        cout << "Invalid index!" << endl;
        return;
    }
    // Inefficient deletion with unnecessary nested loops
    for (int i = idx; i < currentCount - 1; i++) {
        for (int j = i; j < currentCount - 1; j++) {
            dataArray[j] = dataArray[j + 1];
            break;
        }
    }
    currentCount--;
    cout << "Item removed from index " << idx << endl;
}

int findItem(int target) {
    // Inefficient lookup with dummy loop
    for (int i = 0; i < currentCount; i++) {
        for (int k = 0; k < 1; k++) {
            if (dataArray[i] == target) {
                return i;
            }
        }
    }
    return -1;
}

void printAll() {
    cout << "Current List Contents: ";
    for (int i = 0; i <= currentCount; i++) { // Bug: <= causes out of bounds read!
        cout << dataArray[i] << " ";
    }
}

```

```

    }
    cout << endl;
}

void processMatrix() {
    int r = 3, c = 3;
    int m[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
    int t[3][3];

    // Transpose matrix
    for(int i=0; i<3; i++) {
        for(int j=0; j<3; j++) {
            t[j][i] = m[i][j];
        }
    }

    cout << "Transposed Matrix printed raw:" << endl;
    for(int i=0; i<3; i++) {
        for(int j=0; j<3; j++) {
            cout << t[i][j] << " ";
        }
        cout << endl;
    }
}

int main() {
    cout << "--- STARTING LEGACY SUBSYSTEM ---" << endl;
    add_item(10);
    add_item(20);
    add_item(30);
    add_item(40);
    add_item(50);
    add_item(60); // Triggers flawed dynamic resize
    printAll();

    cout << "Found 30 at index: " << findItem(30) << endl;
    remove_item_at(2);
    printAll();

    remove_item_at(99); // Out-of-bounds bug test
    printAll();

    processMatrix();
}

```

```
// Missing delete[] dataArray! Memory Leak on Exit!
return 0;
}

```

8. DETAILED STEP-BY-STEP PROCEDURES

Step 1: Repository Setup and Legacy Code Import

1. Open your terminal or IDE and create a local working directory named `CC105_WEEK5_LastName_FirstName`.
2. Initialize a Git repository inside this folder using `git init`.
3. Create a file named `LegacyLinearSubsystem.cpp` inside a subfolder named `Original_Code/` and copy the legacy code provided in Section 7 into this file. 4. Compile and run `LegacyLinearSubsystem.cpp` using your C++ compiler:

```
g++ -Wall -std=c++17 Original_Code/LegacyLinearSubsystem.cpp
-o Original_Code/legacy_app
./Original_Code/legacy_app

```

5. Observe the runtime warnings, segmentation faults, or buffer read errors that occur during execution.

Step 2: Part A – Initial Peer Code Review & Audit

1. Inspect `LegacyLinearSubsystem.cpp` without altering the file.
2. Complete the **Peer Code Audit Checklist** below by identifying structural problems, improper memory management routines, syntax issues, missing validations, and design anti-patterns.

PART A: Peer Code Audit Checklist Table

Audit Category	Specific Issue Located in Legacy Code	Line Number(s)	Severity (Low / Med / High / Critical)	Impact / Risk

Global State	Use of mutable global variables (<code>dataArray</code> , <code>currentCount</code> , <code>maxCapacity</code>).	Lines 8–10	High	Destroys modularity; prevents multiple array instances; thread-unsafe.
Memory Leak	Dynamic array resizing allocates new memory to <code>temp</code> without calling <code>delete[] dataArray</code> .	Lines 18–22	Critical	Heap memory leak occurs on every capacity double; leads to RAM exhaustion.
Memory Leak	Program terminates without executing <code>delete[] dataArray</code> .	Line 100	High	Heap memory remains unreleased upon program execution completion.
Out-of Bounds	Traversal loop condition <code>i <= currentCount</code> reads uninitialized/garbage heap memory.	Line 58	High	Off-by-one buffer over-read; yields corrupted output or crash.
Out-of Bounds	<code>remove_item_at()</code> lacks upper bound validation checking (<code>idx >= currentCount</code>).	Line 31	Critical	Attempting deletion at invalid indices causes array corruption or crash.
Inefficient Shift	Redundant inner loop inside deletion shifting routine.	Lines 37–41	Medium	Artificially inflates element shift time complexity needlessly.

Audit Category	Specific Issue Located in Legacy Code	Line Number(s)	Severity (Low / Med / High / Critical)	Impact / Risk
Naming Standards	Inconsistent function naming (<code>add_item</code> vs <code>findItem</code> vs <code>printAll</code>).	Multiple	Low	Violates unified C++ naming conventions (camelCase / snake_case).
Hardcoded Sizing	Fixed matrix sizes (<code>3x3</code>) hardcoded in <code>processMatrix()</code> .	Lines 66–68	Medium	Prevents dynamic processing of multi dimensional array grids.

Step 3: Part B – Baseline Unit Testing Suite Execution

1. Construct a formal unit test table containing at least **8 to 10 test cases** to establish a baseline for system behavior.
2. Run each test case against the original un-refactored code and record the results.

PART B: Unit Testing / Test Case Table

Test Case ID	Target Function / Operation	Test Input Vector	Expected Output	Actual Output (Legacy Code)	Pass/Fail	Technical Remarks
TC 01	<code>add_item()</code>	Add <code>10, 20, 30</code> (Normal)	Items added; Count = 3; Capacity = 5.	Items added successfully.	PASS	Normal addition within initial capacity works.

TC 02	add_item()	Add 6 items (Triggers Resize)	Array capacity doubles to 10; all 6 items retained.	Items added, but old array memory leaked in heap.	FAIL	Dynamic allocation leaks previous heap memory buffer.
--------------	------------	-------------------------------	---	---	-------------	---

TC 03	printAll()	Print array with 5 items	Displays indices 0 to 4 cleanly.	Displays 5 valid items plus 1 garbage memory value.	FAIL	Off-by-one error (<code>i <= currentCount</code>) reads past boundary.
TC 04	findItem()	Search for existing target 30	Returns index 2.	Returns index 2.	PASS	Target found despite redundant inner loop.
TC 05	findItem()	Search for missing target 999	Returns -1.	Returns -1.	PASS	Non-existent element lookup handles return safely.
TC 06	remove_item_at()	Delete valid index 2	Item at index 2 removed; array shifts left; count = 4.	Item removed; array shifts left correctly.	PASS	Functional, but execution logic contains redundant loops.

TC 07	<code>remove_item_at()</code>	Delete negative index <code>-5</code>	Rejects input with error message; no state change.	Prints "Invalid index!", state unchanged.	PASS	Negative index check present.
TC 08	<code>remove_item_at()</code>	Delete out-of bounds index <code>99</code>	Rejects input with error message; no state change.	Crash / Corrupts array state (Shifts	FAIL	Lacks upper boundary check (<code>idx >=</code>

				invalid memory).		<code>currentCount</code> .
TC 09	<code>processMatrix()</code>	Transpose <code>3x3</code> Matrix	Transposed grid output correctly formatted.	Displays transposed matrix.	PASS	Functions correctly but lacks dynamic dimension support.
TC 10	Memory Clean	Program Exit	Zero memory leaks reported by Valgrind / AddressSanitizer.	Memory leak reported: lost heap allocation bytes.	FAIL	Lacks destructor/cleanup <code>delete[]</code> call.

Step 4: Part C – Big-O Algorithmic Complexity Analysis

Analyze the time and space complexity of the legacy operations. Fill out the analysis table and provide clear mathematical explanations for best, average, and worst cases.

PART C: Big-O Complexity Analysis Table

Operation	Target Code / Function	Best Case	Average Case	Worst Case	Space Complexity	Mathematical Justification
Unamortized Insert	<code>add_item()</code>	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(1)$ when capacity remains. Worst case $O(n)$ occurs when array reaches
						capacity, requiring reallocation and copying n elements.

Linear Deletion	<code>remove_item_at()</code>	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Deleting the last element takes $O(1)$. Deleting at index 0 requires shifting $n-1$ elements left. The legacy double loop does not change asymptotic complexity, but adds constant factor overhead.
Linear Search	<code>findItem()</code>	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Best case $O(1)$ occurs if target is at index 0. Worst case $O(n)$ occurs if target is at

						index $n-1$ or absent.
--	--	--	--	--	--	------------------------

Array Display	<code>printAll()</code>	$O(n)$	$O(n)$	$O(n)$	$O(1)$	Must sequentially visit and display every element from index 0 to $n-1$.
Matrix Transpose	<code>processMatrix()</code>	$O(R \times C)$	$O(R \times C)$	$O(R \times C)$	$O(R \times C)$	Requires visiting every cell in an $R \times C$ grid via nested loops.

Checkpoint 1: Audit & Testing Validation

CHECKPOINT 1: Before starting code refactoring, present your completed **Peer Code Audit Checklist (Part A)**, **Unit Testing Table (Part B)**, and Big-O Table (Part C) to your laboratory

instructor. *Instructor Signature / Stamp:* _____ *Date:* _____

Step 5: Part D – Code Refactoring and System Optimization

Before	Problem Identify	After	Justification
<code>dataArray = temp</code>	Memory leak because the old array is not freed before replacing the pointer	<code>delete[] dataArray;</code> <code>dataArray = temp;</code>	Frees the old memory and prevent memory leaks during resizing
<code>If (idx < 0)</code>	Missing upper-bound validation. An index like 99 can access outside the array	<code>if (idx < 0 idx >= currentCount)</code>	Prevent invalid or out of bounds index access

for (int i = 0; i <= currentCount; i++)	<= cause the program to read one element beyond the valid array range	for (int i = 0; i < currentCount; i++)	Prevents out of bounds memory access and correctly display all items
---	---	--	--

1. Create a new file named `RefactoredLinearSubsystem.cpp` inside a subfolder named `Refactored_Code/`.
2. Refactor the program to resolve all identified defects. Ensure your refactored implementation adheres to these engineering standards:
 - **Encapsulation:** Wrap data structures into a clean C++ `struct` or `class` (e.g., `DynamicArray` or `LinearLinearArchitecture`).
 - **RAII Memory Hygiene:** Ensure constructors allocate memory dynamically and destructors (`~DynamicArray()`) automatically free heap memory using `delete[]`.
 - **Resource Safety:** Implement proper dynamic array expansion that copies existing elements, frees the old buffer using `delete[]`, and reassigns the pointer correctly without memory leaks.
 - **Input Validation:** Enforce strict bounds checks (`0 ≤ index < currentCount`) on all retrieval, insertion, and deletion methods.
 - **Algorithmic Cleanliness:** Replace redundant loops in deletion and search routines with optimal single-pass routines.
 - **Coding Style:** Enforce consistent naming conventions, clear code formatting, and explanatory documentation.
3. Document your structural changes using the **Before $\rightarrow$ Problem Identified $\rightarrow$ After $\rightarrow$ Justification** format.

Sample Refactoring Documentation Mapping

```

1. Feature: Dynamic Memory Array Resizing
- BEFORE:
int* temp = new int[maxCapacity * 2];
for (int i = 0; i < maxCapacity; i++) temp[i] = dataArray[i];
dataArray = temp; // Memory Leak!
- PROBLEM IDENTIFIED: Old memory allocated to dataArray was never deallocated,
resulting in a heap memory leak upon every resize.
- AFTER:
int* temp = new int[newCapacity];
for (size_t i = 0; i < size; ++i) temp[i] = dataArray[i];
delete[] dataArray; // Release old buffer

```

```
dataArray = temp;
capacity = newCapacity;
- JUSTIFICATION: Prevents heap memory leaks and safely transfers ownership of
dynamically allocated memory buffers.
```

2. Feature: Array Deletion Shifting Logic

```
- BEFORE:
for (int i = idx; i < currentCount - 1; i++) {
for (int j = i; j < currentCount - 1; j++) {
dataArray[j] = dataArray[j + 1]; break;
}
}
- PROBLEM IDENTIFIED: Nested loop structure was redundant and unreadable.
- AFTER:
for (size_t i = idx; i < size - 1; ++i) {
dataArray[i] = dataArray[i + 1];
}
--size;
- JUSTIFICATION: Eliminates unnecessary loop overhead and simplifies element
shifting to a clear  $O(n)$  single-pass loop.
```



Step 6: Part E – Peer Code Review Exchange

1. Pair up with an assigned peer reviewer.
2. Exchange your refactored source code (`RefactoredLinearSubsystem.cpp`).
3. Act as an auditor for your peer's codebase. Review their code against engineering standards and provide feedback using the **Peer Review Evaluation Form** below.

PART E: Peer Review Evaluation Form



```
=====
CC105 PEER CODE AUDIT EVALUATION FORM
=====

= Author Name : [ Name of Code Developer ]
Reviewer Name : [ Your Name / Peer Reviewer ]
Date of Audit : [ Date ]
Repository Link: [ Link to Git Repository / File Path ]

-----
- 1. IDENTIFIED STRENGTHS (At least 3 positive engineering practices observed)
-----
-
```

[1] Enforced RAII by placing delete[] inside the class destructor. [2] Used exception handling / bool error returns for out-of-bound access. [3] Replaced raw magic numbers with const variables and clear template types.

- 2. IDENTIFIED WEAKNESSES / DEFECTS (At least 3 structural or style issues)

-
[1] The processMatrix() function still uses static parameter sizing. [2] Lacks a copy constructor and copy assignment operator (Rule of Three violation). [3] Missing explicit inline comments explaining the matrix transposition loop.

- 3. RECOMMENDED ACTIONABLE IMPROVEMENTS (At least 3 recommendations)

- [1] Implement dynamic template support <typename T> for generic data types.
[2] Add an explicit shrink_to_fit() method to optimize unused allocated memory.
[3] Mark getter functions (e.g., getSize(), Search()) with the `const` modifier.

=====

DEVELOPER ACTION & REJECTION JUSTIFICATION (To be filled by Author)

=====

Recommendation [1]: [ACCEPTED] - Will refactor dynamic array to use template classes.

Recommendation [2]: [PARTIALLY ACCEPTED] - Kept manual capacity controls for Milestone 1 scope.

Recommendation [3]: [ACCEPTED] - Added `const` specifiers to all accessor methods.

Author Signature: _____ Date: _____



Step 7: Part F – Final Regression Testing & Comparison Validation Re-run your full suite of unit test cases against the refactored system. Construct a **Before vs. After Comparison Table** demonstrating that all defects have been resolved.

PART F: Final Testing & Optimization Comparison Table

Test Case ID	Target Feature	Outcome Before Refactoring	Outcome After Refactoring	Engineering Performance Improvement / Observation
TC-01	Basic Element Insert	Functional (Global scope)	Functional (Encapsulated)	Data structures are safely isolated inside a managed class instance.
TC-02	Dynamic Memory Resize	Memory Leak (Heap orphaned)	Zero Leaks (Memory freed)	<code>delete[]</code> frees legacy buffers; heap memory remains stable.
TC-03	Traversal Output	Off-by-one heap over-read	Clean traversal (0... n - 1)	Array bounds checked; garbage values eliminated.
TC-06	Element Deletion	Shifted with redundant loop	Single-pass O(n) shift	Code execution streamlined; eliminates unnecessary loop conditions.

TC-08	Upper Boundary Check	System Crash / Invalid write	Rejects index safely	Enforces bounds checking (<code>idx >= size</code>); throws/ print safe error.
TC-10	Program Cleanup	Leaked dynamic memory	100% Heap Released	AddressSanitizer / Valgrind confirms 0 bytes leaked on exit

Checkpoint 2: Final Verification & Repository Readiness

CHECKPOINT 2: Present your **Refactored Code** (**RefactoredLinearSubsystem.cpp**), **Peer Audit Evaluation Form (Part E)**, and **Final Comparison Table (Part F)** to your instructor. *Instructor Signature / Stamp:* _____ *Date:* _____

9. TESTING AND VALIDATION PROCEDURES

To ensure your refactored C++ code is stable and production-ready for Milestone 1, validate it against these scenarios:

- 1. **Empty State Access:** Verify that calling search, display, or delete operations on an empty dynamic array (`$n=0$`) outputs a clean message rather than crashing or referencing null pointers.
- 2. **Boundary Index Operations:** Test insertion and deletion at index `0` (head boundary) and index `size - 1` (tail boundary).
- 3. **Out-of-Bounds Stress Inputs:** Pass negative indices (`-1`, `-100`) and upper out-of-bounds indices (`size`, `size + 50`) to verify input validation traps.
- 4. **Memory Leak Diagnostics:** Compile your refactored code using GCC AddressSanitizer flags to verify memory safety:

```
g++ -Wall -Wextra -std=c++17 -fsanitize=address
Refactored_Code/RefactoredLinearSubsystem.cpp -o
Refactored_Code/refactored_app ./Refactored_Code/refactored_app
```

Ensure the console reports zero leak errors upon program completion.

10. TROUBLESHOOTING GUIDE

Symptom / Error Message	Root Cause	Technical Resolution
Segmentation fault (core dumped)	Accessing uninitialized pointers or reading past allocated array boundaries (<code>dataArray[i]</code> where <code>\$i</code> $\geq$ <code>\text{capacity}</code>).	Verify that all loop conditions use strict <code><</code> bounds (<code>i < size</code>) and ensure pointers are initialized (<code>nullptr</code>).

double free or corruption (out)	Calling <code>delete[]</code> twice on the same memory address, or copying raw pointers without a custom copy constructor.	Ensure pointer ownership is clear. Nullify pointers (<code>ptr = nullptr</code>) immediately after executing <code>delete[]</code> .
AddressSanitizer: heap-use after-free	Attempting to read or write to a dynamic array buffer after it has been deleted.	Check dynamic array resizing routines; ensure no pointers reference the old buffer after calling <code>delete[]</code> .
Infinite loop / Terminal hangs during search or deletion	Counter variables inside loops are missing increment/decrement steps, or loop conditions do not progress	Inspect <code>for</code> and <code>while</code> loop conditions. Remove redundant inner loop breaks and verify counter modifications.

	toward termination.	
Input stream failure / Skipping menu loops	Passing char/string inputs into <code>int</code> choice variables using <code>cin</code> .	Clear the error state using <code>cin.clear()</code> and flush invalid characters using <code>cin.ignore(numeric_limits<streamsize>::max(), '\n')</code> .

11. EXPECTED RESULTS

Upon completing this activity, your refactored linear data structure subsystem should:

1. Compile cleanly under `-Wall -Wextra` flags without syntax errors or compiler warnings.
2. Manage dynamic memory safety via class constructors and destructors, passing leak detection diagnostics with 0 memory leaks.
3. Handle invalid inputs, boundary conditions, and out-of-bounds index requests without crashing or corrupting memory.
4. Execute array insertions, deletions, linear searches, and matrix transformations using optimal linear routines.
5. Feature an organized Git repository structure containing complete peer audit documentation, test tables, and Big-O efficiency analyses.

12. REQUIRED DELIVERABLES

You must organize and submit the following **10 required deliverables** bundled as your **Module Draft: Peer-Audited C++ Linear Data Architecture Subsystem**:

1. **Original Legacy C++ Source Code:** Unaltered `LegacyLinearSubsystem.cpp` file.
2. **Refactored C++ Source Code:** Complete, clean, and fully commented `RefactoredLinearSubsystem.cpp`.
3. **Peer Code Audit Checklist (Part A):** Completed defect matrix identifying initial code quality issues.
4. **Unit Testing Suite Table (Part B):** Complete test case matrix with 8–10 test cases including inputs and outputs.
5. **Big-O Complexity Analysis Matrix (Part C):** Complete complexity analysis breakdown table with mathematical explanations.
6. **Before-and-After Refactoring Documentation (Part D):** Structured listing of specific code fixes using the *Before* $\rightarrow$ *Problem* $\rightarrow$ *After* $\rightarrow$ *Justification* format.
7. **Completed Peer Review Form (Part E):** Peer audit evaluation sheet signed by both auditor and code author.
8. **Final Testing & Regression Comparison Matrix (Part F):** Validation table comparing baseline execution against refactored code performance.
9. **Compilation & Execution Screenshots:** High-resolution screenshots verifying clean compilation and runtime test execution.

10. **Milestone 1 Repository Preparation Evidence:** Organized local repository directory with a comprehensive `README.md` file detailing project layout and build instructions.

13. GUIDE QUESTIONS

Answer the following questions in your final report:

1. **Data Structure Selection & Trade-Offs:** Why are dynamic arrays preferred over fixed-size static arrays for unpredictable batch workloads, and what hidden computational costs occur during memory reallocations?
2. **Memory Hygiene & Lifecycle:** Explain the role of the C++ destructor (`~DynamicArray()`) in managing heap allocation safety. What system failure occurs if `delete[]` is omitted in long-running C++ applications?
3. **Defensive Programming & Bounds Enforcement:** Contrast the behavior of raw array indexing (`array[i]`) with bounds-checked operations. Why is missing upper-bound validation (`idx >= size`) dangerous in security-critical codebases?
4. **Algorithmic Overhead & Code Smells:** How did eliminating redundant nested loops inside the array deletion routine change the runtime performance of the system?
5. **Value of Peer Code Audits:** Why are structured peer code reviews essential in commercial software development prior to merging feature branches into production repositories?

GUIDE QUESTIONS (Answer)

1. **Data Structure Selection & Trade-Offs:** Dynamic arrays provide fast $O(1)$ random access via index arithmetic, making them superior to fixed static arrays when workload volume is unpredictable. However, dynamic array reallocations carry hidden computational overheads: when capacity limits are hit, allocating new heap buffers, copying elements, and freeing old memory results in temporary $O(n)$ operational latency spikes.
2. **Memory Hygiene & Lifecycle:** In C++, class destructors (`~DynamicArray()`) automatically manage cleanup when objects go out of scope. Omitting `delete[]` in long-running processes results in cumulative heap allocation leaks that gradually exhaust system RAM, ultimately causing operating system instability or abrupt runtime crashes.
3. **Defensive Programming & Bounds Enforcement:** Raw indexing (`array[i]`) performs direct pointer offsets without checking allocated boundaries. Lacking upper-bound checks (`idx >= size`) enables out-of-bounds writes or reads, exposing security vulnerabilities like buffer overflow exploits or arbitrary memory corruption.

4. Algorithmic Overhead & Code Smells: Removing redundant nested loops inside deletion logic reduced constant factor execution overhead while keeping asymptotic complexity at $O(n)$. This clean single-pass routine minimizes CPU cycles spent per element shift operation.
5. Value of Peer Code Audits: Peer audits ensure software quality by identifying logic bugs, memory leaks, and architectural flaws before code reaches production. They maintain clean design standards across teams and reduce technical debt

14. REFLECTION QUESTIONS

Include thoughtful responses to the following reflection prompts in your submission:

1. Experiential Reflection: What was the most challenging defect to locate inside the legacy codebase, and how did systematic unit testing assist you in pinpointing the root cause compared to random debugging?

The trickiest bug was the off-by-one condition ($i \leq \text{currentCount}$) in `printAll()` because it ran without crashing on standard tests, outputting adjacent stack/heap noise. Structured unit test tables isolated boundary inputs, making the over-read easy to catch compared to informal debugging.

2. Architectural Evaluation: Knowing what you now know about dynamic array reallocations and shift-based element deletions, what linear architecture alternative (e.g., Linked Lists, Queues, Stacks) would you recommend for systems that require frequent mid-list insertions without reallocating buffers?

For systems needing frequent middle insertions without dynamic memory reallocations. Singly or Double Linked Lists are recommended. Linked lists permit insertion/deletion in $O(1)$ time once the target node is located, bypassing the element-shifting overhead completely.

3. Professional Growth: How has evaluating another programmer's code during the Peer Code Review altered your approach to writing clean, well-documented C++ code for your upcoming Milestone 1 project?

Evaluating other programmers is the most respectful thing that the other programmers can do because with this evaluation programmers can improve their strengths and weaknesses based on the other programmers recommendations and also help them with their projects with the evaluation other programmers can evaluate on what to improve.

15. SUBMISSION INSTRUCTIONS AND REPOSITORY FORMAT

You must structure your submission as a clean Git repository folder. Compress the root directory into a single `.zip` file and upload it to the official Learning Management System (LMS) before the deadline.

Required Directory Folder Structure

```
t
CC105_WEEK5_LastName_FirstName/
|
|— Original_Code/
|   |— LegacyLinearSubsystem.cpp # Raw, unaltered legacy codebase |
|— Refactored_Code/
|   |— DynamicArray.h # Header file with class/struct declarations | |—
DynamicArray.cpp # Implementation file with member functions | |—
RefactoredLinearSubsystem.cpp # Refactored main driver program |
|— Testing/
|   |— Baseline_UnitTest_Table.pdf # Completed Part B test suite table | |—
Final_Regression_Table.pdf # Completed Part F comparison table |
|— Peer_Audit/
|   |— Peer_Audit_Checklist.pdf # Completed Part A audit checklist
|   |— Peer_Evaluation_Signed.pdf # Completed Part E peer audit review form |
|— Screenshots/
|   |— 01_Legacy_Compilation_Errors.png # Screen capture of initial build issues | |—
02_Refactored_Clean_Build.png # Screen capture of zero-warning compilation | |—
03_Memory_Sanitizer_Output.png # Screen capture confirming zero memory leaks |
|— Documentation/
|   |— BigO_Complexity_Analysis.pdf # Completed Part C complexity analysis |
|— Refactoring_Documentation.pdf # Completed Part D Before/After mapping |
|— Guide_and_Reflection_Answers.pdf # Written answers to Sections 13 & 14 |
|— README.md # Comprehensive repository guide
```



`README.md` Requirements

Your repository root must contain a professional `README.md` file featuring: *

****Project Title & Student Metadata:**** Name, Student ID, Course Section, Instructor, Date.

- * **Executive Summary:** Overview of the linear subsystem, audit findings, and refactoring strategies applied.
- * **Build & Compilation Commands:** Explicit terminal instructions for compiling and executing the code across platforms.
- * **Module Architecture Overview:** Description of classes, member variables, and functions.
- * **Milestone 1 Preparedness Statement:** Brief declaration explaining how this synthesized linear subsystem prepares your codebase for the upcoming Milestone 1 submission.

16. GRADING RUBRIC – 100 TOTAL POINTS

Assessment Criteria	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)	Score
C++ Program Functionality & Correctness (25 Points) Flawlessly refactored C++ code. Zero logic errors, zero memory leaks (verified via sanitizer), perfect dynamic allocation and bounds checking across all edge cases. (22–25 pts) Code is fully functional with minor code style issues. Resolves major memory leaks and logical bugs; handles standard inputs cleanly. (18–21 pts) Program runs but contains non-critical bugs, edge-case failures, or minor unreleased memory allocations. (14–17 pts) Program fails to compile, crashes frequently, or fails to fix legacy dynamic memory leaks. (0–13 pts)	Code Quality, Refactoring & Modularity (20 Points) Exemplary code structure. Encapsulated in clean classes/structs; adheres to C++ naming standards; zero global variables; excellent documentation and RAII dynamic memory safety. (18–20 pts) Clean refactored code. Good modularization into functions/classes; readable variable naming; appropriate comments and memory cleanup. (14–17 pts) Partial refactoring. Code relies on monolithic functions or minor global states; inconsistent variable naming and minimal inline comments. (10–13 pts) Poor code structure. Unorganized monolithic code; retains global states, dangerous raw pointers, and magic numbers. (0–9 pts)	Unit Testing & Validation Suite (20 Points) Comprehensive unit test suite (≥ 10 test cases). Accurately covers normal, boundary, out-of-bounds, and empty inputs with detailed actual vs. expected outputs. (18–20 pts) Complete test suite (8–10 test cases) covering normal and boundary conditions with clear pass/fail remarks. (14–17 pts) Test suite lacks sufficient test cases (< 8), or misses boundary/out-of-bounds inputs. Remarks are vague. (10–13 pts) Incomplete test table. Test cases are missing, inaccurate, or fail to test critical linear operations. (0–9 pts)	Code Quality, Refactoring & Modularity (20 Points) Exemplary code structure. Encapsulated in clean classes/structs; adheres to C++ naming standards; zero global variables; excellent documentation and RAII dynamic memory safety. (18–20 pts) Clean refactored code. Good modularization into functions/classes; readable variable naming; appropriate comments and memory cleanup. (14–17 pts) Partial refactoring. Code relies on monolithic functions or minor global states; inconsistent variable naming and minimal inline comments. (10–13 pts) Poor code structure. Unorganized monolithic code; retains global states, dangerous raw pointers, and magic numbers. (0–9 pts)	Unit Testing & Validation Suite (20 Points) Comprehensive unit test suite (≥ 10 test cases). Accurately covers normal, boundary, out-of-bounds, and empty inputs with detailed actual vs. expected outputs. (18–20 pts) Complete test suite (8–10 test cases) covering normal and boundary conditions with clear pass/fail remarks. (14–17 pts) Test suite lacks sufficient test cases (< 8), or misses boundary/out-of-bounds inputs. Remarks are vague. (10–13 pts) Incomplete test table. Test cases are missing, inaccurate, or fail to test critical linear operations. (0–9 pts)	Code Quality, Refactoring & Modularity (20 Points) Exemplary code structure. Encapsulated in clean classes/structs; adheres to C++ naming standards; zero global variables; excellent documentation and RAII dynamic memory safety. (18–20 pts) Clean refactored code. Good modularization into functions/classes; readable variable naming; appropriate comments and memory cleanup. (14–17 pts) Partial refactoring. Code relies on monolithic functions or minor global states; inconsistent variable naming and minimal inline comments. (10–13 pts) Poor code structure. Unorganized monolithic code; retains global states, dangerous raw pointers, and magic numbers. (0–9 pts)

| **Peer Code Audit & Defense Documentation**
(15 Points)* | Thorough Part A checklist and Part E evaluation. Identifies all major code defects with actionable feedback; provides valid justifications for author decisions.
(14-15 pts)* | Complete audit checklist and peer review form. Identifies major code flaws and offers constructive recommendations.
(11-13 pts)* | Audit forms are partially completed. Misses several obvious code flaws or provides superficial recommendations.
(8-10 pts)* | Peer audit forms are missing, incomplete, un-signed, or lack meaningful evaluation details.
(0-7 pts)* | **/15** |

| **Big-O Algorithmic Complexity Analysis**
(10 Points)* | Flawless Big-O analysis matrix for time (best, avg, worst) and space complexities with precise mathematical justifications for all routines.
(9-10 pts)* | Accurate Big-O analysis matrix with minor omissions in mathematical explanations.
(7-8 pts)* | Big-O complexities identified, but contains errors in best/worst case distinctions or weak explanations.
(5-6 pts)* | Big-O analysis is inaccurate, missing critical operations, or lacks mathematical reasoning.
(0-4 pts)* | **/10** | |

Deliverables, Screenshots & Reflection
(10 Points)* | All 10 deliverables present in the specified Git directory layout. Excellent screenshots, detailed `README.md`, and deep, well-reasoned answers to guide/reflection questions.
(9-10 pts)* | All major deliverables submitted in the correct directory format with clean execution screenshots and complete reflection responses.
(7-8 pts)* | Missing 1-2 minor deliverables or screenshots. Answers to guide/reflection questions are brief or lack depth.
(5-6 pts)* | Disorganized submission. Missing multiple required deliverables, screenshots, or reflection responses.
(0-4 pts)* | **/10** | |

TOTAL SCORE | | | | **/100** |

17. CHECKPOINTS AND INSTRUCTOR SIGN-OFF

To maintain academic progression during the laboratory session, you must obtain instructor sign-offs at two key milestones before final LMS submission:

* **Checkpoint 1 (Initial Code Audit & Test Planning):** Present completed Parts A, B, and C tables to verify that all legacy defects and complexities are documented prior to refactoring.

* **Checkpoint 2 (Refactored Subsystem & Peer Review Verification):** Present your running, leak-free C++ code (`RefactoredLinearSubsystem.cpp`), completed Part E peer review forms, and Part F validation results.

18. ACADEMIC INTEGRITY AND CODE DEFENSE POLICY

* **Institutional Ethics Statement:** In alignment with National Teachers College core values of **Integridad** and **Kahusayan**, all source code, audit tables, and

reflection responses submitted must represent the original work of the student or student pair.

* **Plagiarism & Code Sharing Policy:** Copying source code from online repositories, generative AI tools without attribution, or peer submissions without performing the actual audit process is strictly prohibited and constitutes academic dishonesty under the NTC Student Handbook.

* **Practical Oral Code Defense:** The instructor reserves the right to conduct an impromptu **1-on-1 Code Defense** during submission checking. You must be prepared to explain line-by-line logic, pointer operations, memory management choices, and Big-O derivations.

* **Grade Adjustment Policy:** Submissions containing copied code or code that the student cannot explain, justify, or modify live during defense will receive zero (0) points for the C++ Program Functionality and Code Quality criteria, subject to formal disciplinary review.

